



Module 24-1

SystemVerilog for Verification: Random Stimulus

cādence[®]

This page does not contain notes.

Module Objectives

In this module, you

- Identify the benefits of using random data generation in verification
- Create random data using the randomize method
- Use constraints to control randomization



This page does not contain notes.

CPU Coverage Test Case



Declare opcode and register values as enumerated types.

Assign values for driving into DUT.

This Simplified 8-bit CPU has:

- 7 immediate instructions:
 - ADDI, SUBI, ANDI, XORI, JMP, JMPC, CALL
- 4 registers:
 - REG0, REG1, REG2, REG3

In the double-byte format as shown:

Problem: How to test the widest combinations of opcode, register identifier and data?

Solution 1: Using Looped CPU Instruction Stimulus

Solution 2: Applying SV Randomization and Constraints

CPU Instruction Stimulus

```
typedef enum bit[2:0] {ADDI, SUBI, ANDI, XORI,
                        JMP, JMPC, CALL} op_t;
typedef enum bit[1:0] {REG0, REG1, REG2, REG3} regs_t;

op_t      opc;
regs_t    regs;
logic[7:0] data;

initial begin
    // generate stimulus
    opc = ADDI;
    regs = REG0;
    data = 5;
    // drive into DUT
    ...
end
```

15	14	13	12	11	10	9	8
X	X	X	opcode				
7	6	5	4	3	2	1	0
-	-	-	data		-	-	-

Let us take a look at why you might want to generate random stimulus and constrain it to meaningful sets of values.

To do this, we will use a simplified example of an 8-bit CPU on which we want to test the defined immediate instruction set with four internal registers.

Stimulus is sent to the CPU as an 8-bit data field, a 2-bit register field to address four registers, and a 3-bit opcode field to specify one of the seven immediate instructions.

Solution 1: Looped CPU Instruction Stimulus

```

typedef enum bit[2:0] {ADDI, SUBI, ANDI, XORI, JMP, JMPC,
CALL} op_t;
typedef enum bit[1:0] {REG0, REG1, REG2, REG3} regs_t;

op_t      opc;
regs_t    regs;
logic[7:0] data;

initial begin
  opc = opc.first();
  regs = regs.first();
  repeat(opc.num()) begin
    repeat (regs.num()) begin
      for (data=0; data<255; ++data)
        @(posedge clk);
      regs=regs.next();
    end
    opc = opc.next();
  end
  ...

```

```

ADDI REG0 00 - FF
ADDI REG1 00 - FF
ADDI REG2 00 - FF
ADDI REG3 00 - FF
SUBI REG0 00 - FF
SUBI REG1 00 - FF
SUBI REG2 00 - FF
SUBI REG3 00 - FF
ANDI REG0 00 - FF
...

```

Use nested loops to index through 7168 combinations

A simple approach uses nested loops to generate all possible values over the ranges of the three variables.

The outer loop iterates over the opcode values, the middle loop over the register identifiers, and the inner loop over the range of data values. This generates 7168 separate combinations of values.

Analyzing Looped Stimulus Coverage

- Structured stimulus does not thoroughly test CPU functionality.
- It misses most transitions between value combinations.
 - What if `ADDI` followed by `JMP` causes an out-of-range error?

Structural Metrics look good!

	REG0	REG1	REG2	REG3
ADDI	✓	✓	✓	✓
SUBI	✓	✓	✓	✓
ANDI	✓	✓	✓	✓
XORI	✓	✓	✓	✓
JMP	✓	✓	✓	✓
JMPC	✓	✓	✓	✓

Functional Metrics do not!

ADDI:ADDI	✓
ADDI:SUBI	✓
ADDI:ANDI	✗
ADDI:XORI	✗
ADDI:JMP	✗
ADDI:JMPC	✗
...	

REG0:REG1	✓
REG1:REG2	✓
REG2:REG3	✓
REG0:REG2	✗
REG1:REG3	✗
...	

We then measure the structural and functional coverage of the 7168 patterns.

Structural metrics look good – we have applied every possible value to each variable. Functional metrics do not look good. We have missed most of the transitions between values.

Applying all such transitions requires a much more complex set of stimulus loops!

What we really want to do is to generate random sequences of operations, registers and data, and constrain those random combinations to meaningful combinations.

Solution 2: Employ SV Randomization and Constraints

SystemVerilog provides a new set of constructs for generating and constraining random variables.

- Random generation allows minimal code to generate many data sets.
- Constraints restrict the data set to meaningful data.
- You can apply randomization (with constraints) to:
 - Local variables:
 - "Scope" randomization (this module).
 - Class properties:
 - Classes covered later



Randomization allows you to produce large amounts of stimulus with a minimal amount of code. This lets you more thoroughly and effectively verify the design.

However, full, unrestricted randomization is rarely useful. Control over generated values is required to maintain dependencies between different variables, exclude illegal or undesired values or just restrict data to specific areas of interest.

With randomization constraints, you can restrict the generation and probability of random values to specific values, ranges or areas of interest.

SystemVerilog offers two forms of randomization:

- Randomization of local variables (called scope randomization) covered in this module.
- Randomization of class properties, covered in a later module.

What Are Pseudo-Random Number (PRN) Generators?



PRN Generators generate random sequences algorithmically which are controlled by initial values called as seeds.

Pure random number sequences are useless for verification:

- Sequence must be repeatable.
- Allows debug-fix-debug cycle.

Verification uses *pseudo-random* sequences:

- Controlled by seed(s):
 - Using the *same* seed generates the *same* sequence.
 - Using a *different* seed generates a *different* sequence.

Random

Sim1:	2	5	1	1	6	6	7	0	7	4	3	8
Sim2:	6	2	4	5	4	2	6	0	4	1	5	3
Sim3:	4	4	1	7	3	3	2	8	5	7	3	2

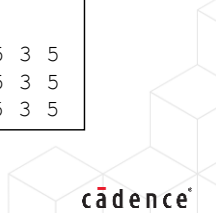
Fixed error cannot be verified,
if sequence does not repeat

Pseudo-random

Seed = 4												
Sim1:	4	5	2	1	0	5	1	3	2	1	8	7
Sim2:	4	5	2	1	0	5	1	3	2	1	8	7
Sim3:	4	5	2	1	0	5	1	3	2	1	8	7
Seed = 7												
Sim1:	2	5	6	1	3	2	1	7	4	5	3	5
Sim2:	2	5	6	1	3	2	1	7	4	5	3	5
Sim3:	2	5	6	1	3	2	1	7	4	5	3	5

Pure randomization is useless for verification. Pure randomization creates different data patterns for every simulation run. Hence if you find, debug and define a fix for an error, you cannot check the fix because you cannot repeat the sequence of random values which originally caused the error. The sequence of values created by multiple randomization must be repeatable to successfully debug the design.

For verification and debug, we need a repeatable (pseudo-random) sequence. These sequences are generated algorithmically. The algorithms are controlled by starting values called seeds. The seed is input to the algorithm to generate the first random value. This random value then becomes the input to the algorithm to generate the next random value, and so on. Hence, for a given seed, the sequence of multiple random values is always the same, but the sequence can be changed simply by using a different seed.



randomize() : Randomizing Scope Variables



IEEE 1800-2012 18.7

- The `randomize()` function generates random values on variables:
 - Returns 1 on success:
 - Variables can be randomized
 - Constraints can be met
 - Otherwise returns 0
- You can randomize integral scalar and array variables.
- `randomize()` is defined in a built-in package called `std`:
 - Automatically imported
 - Singular, Integral and Variable restrictions

```
//type declarations as before
opc_t      opc;
regs_t     regs;
logic[7:0] data;

int ok;

initial begin
    repeat(7168) begin
        ok = randomize(opc, regs, data);
        @(posedge clk);
    end
    ...
```

Function randomizes variables passed as arguments

It is really easy to generate random values on scope variables. You simply pass the variables as arguments to a built-in function called `randomize()`. The function returns an `int` value, which is 1 for successful randomization and 0 for failure. Typically, randomization will fail if constraints conflict and prevent the random number generator reaching a valid solution. Every time the function is called, random values are applied to the variable arguments.

You can only randomize singular integral variables.

The singular restriction prevents you from randomizing unpacked arrays or unpacked structures. The integral restriction prevents you from randomizing real numbers and the variable restriction prevents you from randomizing any nets.

The `randomize()` function is declared in a predefined package called `std`, which is automatically and implicitly imported into every SystemVerilog module. If you wish, you can use the full declarative path for the function `std::randomize()`, but the package reference is optional.

What Is a Constraint Block?



The *constraint_block* is a list of expression statements that restrict the range of a variable or define relations between variables.

- Use the *with* clause to attach a constraint block to the *randomize* method:

```
typedef enum bit[2:0] {ADDI, SUBI, ANDI, XORI, JMP, JMPC, CALL} op_t;
typedef enum bit[1:0] {REG0, REG1, REG2, REG3} regs_t;

op_t      opc;
regs_t    regs;
logic[7:0] data;
int ok;
initial begin
    ok = randomize(data) with { data>=32; data<=126; };

    ok = randomize(opc)  with { opc inside {[ADDI:ANDI], JMP, JMPC}; };

    ok = randomize(regs) with { regs dist { [REG0:REG1]:=2, [REG2:REG3]:=1 }; };
    ...
end
```

Relational: data between 32 and 126

List: opc in range ADDI to ANDI or JMP or JMPC

Distribution: REG0-REG1 twice as likely as REG2-REG3

9

© Cadence Design Systems, Inc. All rights reserved.

cadence

So far our randomization has been unconstrained, with every possible value of a variable equally probable. Constraints allow us to define a subset of values or different probability distribution. They are rules which the randomizer must follow to generate a value.

A constraint block can be attached to the *randomize* method using the *with* clause, and enclosing constraint expressions in curly brackets `{ }`. Every constraint expression within the brackets must be terminated in a semicolon.

Constraint expressions can be relational, as in the *data* constraint.

Constraint expressions can define a list of values and value ranges using the *inside* operator, as in the *opc* constraint. The choice of *opc* is limited to the values in the list. Be careful with syntax – the *inside* operator uses curly brackets to define the value list, but the range and value items within the list are comma separated.

Constraints can also change the probability of values using the *dist* operator. *dist* is an enhancement of *inside*, where the values or ranges in the list are given weights – relative probabilities – such as in the *regs* constraint. Here, values *REG0* and *REG1* each has a weight of 2, whereas *REG2* and *REG3* each has a weight of 1. Hence *REG0* has a probability of 2 (its relative weight) divided by 6 (sum of all value weights) i.e., 1/3. The default weight is 1.

A constraint expression can be almost any integral expression. It cannot have side-effects such as using assignment operators. You can use functions in constraints, but there are restrictions. Please refer to the *SystemVerilog Language Reference Manual (LRM)*.

How to Write Conditional Constraints



Conditional constraints allow you to select between different sets of constraints, depending on the value of another variable.

There are two ways of defining conditional constraints:

1. Implication, using the `->` operator
2. if-else, using the `if...else` construct

```
logic[7:0] data;
typedef enum bit[1:0] {SMALL, MEDIUM, LARGE, XL} mode_t;
mode_t mode;

...
ok = randomize(data) with
    {mode == SMALL -> data < 100;
     mode == LARGE -> data > 200;};

ok = randomize(data) with
    { if (mode == SMALL) data < 100; else
      if (mode == LARGE) data > 200; };
...
```

If mode is SMALL, data constraint is `<100`

If mode is LARGE, data constraint is `>200`

If mode is neither SMALL nor LARGE, data is unconstrained

Same constraints with if/else

10

© Cadence Design Systems, Inc. All rights reserved.

cadence

Conditional constraints allow you to select between different sets of constraints, depending on the value of another variable.

There are two ways of defining conditional constraints:

- Implication, using the `->` implication operator from SystemVerilog concurrent assertion syntax:
 - `expression -> constraint_set`
- If-else (using the if-else construct):
 - `if ( expression ) constraint_set [ else constraint_set ]`

Both forms are equivalent, i.e., choice of form is on syntax preference, not functionality.

If no condition in a conditional constraint block is true, then the variables in the block are unconstrained.

Multiple constraints can be defined in each conditional branch by enclosing them in an additional layer of curly brackets.

Module Summary

With simple randomization features, you can:

- Generate large amounts of stimulus data from the compact code:
 - Run longer simulations with more stimulus that more thoroughly tests the design
 - Spend more of your own time crafting directed tests of corner cases
- Constrain the values of the random variables:
 - To create legal stimulus
 - To explore areas of interest
 - To conditionally switch between modes of operation



You can use these SystemVerilog randomization features to easily generate large amounts of stimulus from small amounts of code.

Random stimulus is usually used in conjunction with directed testing. Randomization can test the more easily controlled and observed functionality of the design, while allowing you to spend your time crafting directed tests to verify the less easily controlled and observed functionality.



Module 24-2

SystemVerilog for Verification: Basic Classes

cādence®

This page does not contain notes.

Module Objectives

In this module, you

- Identify the basic principles and benefits of SystemVerilog classes
- Declare simple classes, properties and methods
- Use explicit constructors to initialize properties
- Define static properties and methods
- Create aggregate classes to group class instances
- Apply inheritance and examine issues with constructors
- Use data encapsulation and information hiding for robust classes



This page does not contain notes.

What Is Object-Oriented Programming?



Object-Oriented Programming is a programming paradigm that focuses on objects (as opposed to process).

Data-oriented rather than procedure-oriented

- Focuses on objects instead of actions
- Defines classes to characterize the objects
- Classes declare and own:
 - Properties (AKA fields or data)
 - Methods (AKA subprograms or tasks/functions)

class Student

PROPERTIES

- grade
- name
- number

BEHAVIORS

- eat
- play
- sleep
- study



A+
Jack
1234



A+
John
2341



A+
Jane
3412



A+
Jill
4123

Object-Oriented Programming is a programming paradigm that focuses on *objects*, as apposed to *processes*.

It is data-oriented rather than procedure-oriented.

It focuses on objects instead of actions.

It partitions a programming problem into object types, that we refer to as “classes.”

- An object of the class has properties, the collective value of which define the object’s state.
 - The class definition can choose to individually hide or make visible its properties.
- An object of the class has methods, that collectively define how the object is created, how it interacts with other objects, and how it reacts to applications that use it.
- The class definition can choose to individually hide or make visible its methods
 - Objects interact with other objects and applications only through visible properties and methods. The visible properties and method are termed the “public interface.”

For example, let us imagine an object type that we name “student.” All objects of that type will have the properties of that type, and exhibit the behaviors of that type. Different objects can have different property values, and those values can effect their methods, but all objects of that type will have the properties of that type, and exhibit the behaviors of that type.

What Is a Class?



A class is a type that includes data and subroutines (functions and tasks) that operate on those data.

```
module cmod;

    class myclass;
        int number;
    endclass

    ...
```

Class Declaration in a module

```
package myclasses;

    class frame;
        logic [4:0] addr;
        logic [7:0] payload;
        bit parity;
    endclass

    ...
```

Class Declaration in a Package

- A class is a user-defined data type:
 - Declared in a module, package, interface or other design element
- Classes define data and subroutines (tasks/functions) that operate on the data.
- Class objects can be dynamically created and deleted during simulation.
- Used in Object-Oriented (OO) programming for testbenches and simulation models.
- OO features which are supported include the following:
 - Abstract data modeling
 - Inheritance
 - Data hiding and encapsulation
 - Polymorphism

A class is a user-defined type that includes data items of different types (like a `struct`) but also contains tasks/functions to access and manipulate the data items.

Variables of class types are dynamic, meaning they can be created and deleted during a simulation run, which makes them extremely useful for verification. For example, we can use class variables (handles) to create a random number of data items for stimulus, drive the data into the design, capture the output data and once the data is checked and verified, delete both input and output items to save memory space.

Classes allow the creation of generic, reusable objects that can be later extended, inherited, constrained, overridden, enabled, disabled, and merged with or separated from other objects.

A class declaration is like any other type declaration and must be placed inside a design element (module, program, interface or package).



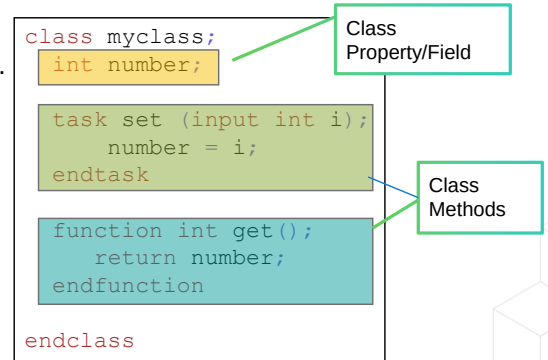
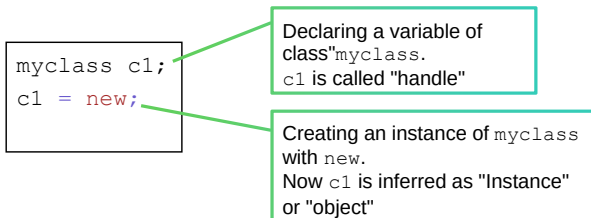
What Is a Class Object?



A class defines a data type. An object is an instance of that class. An object is used by first declaring a variable of that class type (that holds an object handle) and then creating an object of that class (using the **new** function) and assigning it to the variable.

The class type declaration declares class members:

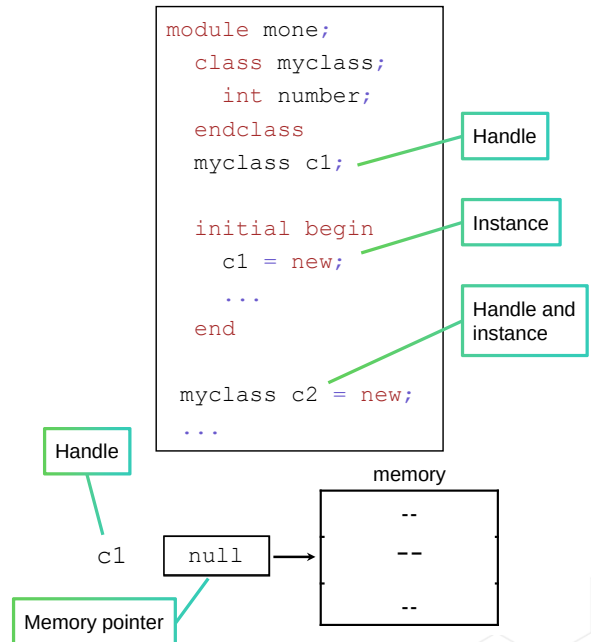
- Data items, called properties or fields.
- Tasks or functions, which operate on data items, called methods.



As we know, class is an user-defined data type, and can contain properties and methods as shown. An object is one instance of the class type. An object is an area of memory allocated for your program and associated with a type. In this example, we have a class called, "my class". It has a property called "number". It also has two methods. on the left side, you are declaring a variable C1 of type, "myclass". Now, C1 is called as handle. We cannot access class properties or methods with this handle, since we have not yet created an instance. In the next line, when you say, "new", then, this creates an instance to the class. we will look more into this in upcoming slides.

Variables of the Class Type

- A variable of a class type is called a handle:
 - The uninitialized value is `null`.
- A class instance must be created for handle:
 - Using function `new`.
 - Assigns a pointer to an area of memory holding the class instance.
 - A handle can be declared and created at the same time:
 - Is no longer used.
 - Is assigned `null`.
 - Automatic “garbage collection.”



17 © Cadence Design Systems, Inc. All rights reserved.

cadence

A class is defined as a data type. The declaration contains data items (class properties) and optionally subroutines to manipulate properties (methods).

To use a class, you declare a variable of the class type, called a handle. The handle contains a pointer to a location in memory which holds the class data. An uninitialized handle has the value `null`. A class instance is created by calling a special class method called `new`, which allocates an area of memory for the class instance and places a pointer to that area in the class handle. The `new` method is called the class constructor.

A class instance can be declared and constructed in one line or the handle declared and the instance constructed later in a procedural statement.

In Object-Oriented languages such as C++, there is a corresponding destructor method, which must be called when the instance is no longer used. In such languages, memory management (construction of new instances and recycling of used instances) is solely under the user control.

In SystemVerilog, class memory management is handled automatically. There is no destructor method and the recycling of used instances is handled by the simulator. Once created, an instance will persist until either:

- The instance is no longer used. For example, if its handle has been used to create a new instance, or the region where the instance was created no longer exists. For example, for a instance declared in a task, the instance can be recycled when the task returns.
- All handles on the instance are explicitly assigned the value `null`. This is not necessary and rarely done.

Accessing Object Members by Using "dot" (.) Operator

- Class members are accessed using "."
- Class properties can be accessed directly or via class methods.

Direct access

```
myclass c1 = new;
...
initial begin
    c1.number = 4;
    $display("c1: %0d", c1.number);
end
```

```
class myclass;
    int number;

    task set (input int i);
        number = i;
    endtask

    function int get();
        return number;
    endfunction

endclass
```

Method access

```
myclass c2 = new;
...
initial begin
    c2.set(3);
    $display("c2: %d", c2.get());
end
```



The class type declaration contains the declaration of class members. Class members can be data items, called properties or fields. Only variable declarations are allowed as class properties; net data types are not allowed. Class members can also be tasks or functions, called methods, which manipulate the class properties. You reference the members of a class instance by using the "dot" (.) notation.

By default, you have full and unrestricted access to all members of a class. We will look at restricting access later in this module.

Quick Reference Guide: C++ Object Pointers and SystemVerilog Object Handles



Operation	C++ Pointer	SystemVerilog Object Handle
Arithmetic operations, e.g., incrementing	allowed	not allowed
For arbitrary data types	allowed	not allowed
Assignment to an address of a data type	allowed	not allowed
Access via a null pointer	error	error*
Casting	unlimited	limited
Default value	undefined	null
Memory management	manual	automatic

SystemVerilog restricts what you can do with class handles.

- This is to reduce the likelihood of user error.

Note: SystemVerilog performs automatic memory management.



Compared to other Object-Oriented languages such as C++, SystemVerilog heavily restricts what you can do with a class handle. Things you cannot do include:

- Arithmetic operations, for example incrementing and decrementing
- Using with any arbitrary data type
- Assigning the address of a variable
- Dereferencing
- Casting to any arbitrary pointer type

A simpler class handle implementation leads to fewer issues and errors in using classes.

A key benefit of the SystemVerilog Object-Oriented implementation is that memory management is automatic. The simulator is responsible for recycling used class instances and for preventing memory leaks – where class instances are repeatedly created without being recycled, leading to a gradual reduction in free system memory and performance, and an eventual software crash.

SystemVerilog is closer to modern Object-Oriented languages such as Java in this respect.

External Method Declaration



Purely for convenience and readability, you can declare the implementation of a class method outside of the class declaration using the `extern` keyword.

- Define the methods outside the class declaration.
- Define the method prototype in the class, prefixed by the keyword `extern`:
 - First line of method, identifying type, name and arguments
- Implement the method outside the class declaration, but in same scope.
 - Implementation must exactly match prototype
 - Link to prototype using the scope resolution operator (`::`):
`class_name::method_name`

```
class myclass;
  int number;

  task set (input int i);
    number = i;
  endtask

  extern function int get();
endclass
```

```
function int myclass::get();
  return number;
endfunction
```

```
myclass c2 = new;

initial begin
  c2.set(3);
  $display("c2: %d", c2.get());
end
```

Purely for convenience and readability, you can declare the implementation of a class method outside of the class declaration using the `extern` keyword.

This gives the class user a quick, compact description of the class properties and methods, uncluttered by the full implementation of every method.

To use external method declarations, you declare the method prototype, prefixed with the keyword `extern`, in the class declaration. The method prototype is the first line of the method declaration, without the method code body. This identifies the type of method (function or task), the return type (for a function), the method name and the arguments. The full method description, including the code body, is then declared outside the class declaration. However, as the method is a class member, not a subroutine of the external scope, you must link its declaration to the class by prefixing the method name with the class name, using the scope resolution operator.

External (or out-of-block) method declarations are purely for readability and have no effect on simulation.

What Is a Constructor Method?



The `new` function which is used to create a class instance and to initialize an instance is called the class constructor.

Method `new` is a special class method called a constructor:

- Defined by default for all classes.
- You can explicitly define it, for example, to initialize class properties.
- Just like any other class function *except* `new` has no return type.

Default constructor

```
class defcon;
  integer number;
endclass

defcon c1 = new;
// c1.number = x
```

Explicit constructor

```
class expcon;
  integer number;

  function new();
    number = 5;
  endfunction

endclass

expcon c2 = new;
// c2.number = 5
```

```
class argcon;
  integer number;

  function new(input int ai);
    number = ai;
  endfunction

endclass

argcon c3 = new(3);
// c3.number = 3
```

Explicit constructor with arguments

By default, class properties initialize to their default values, e.g., `x` for 4-state logic types, `0` for 2-state logic types, etc.

An explicit constructor can be implemented to initialize properties as required. An explicit constructor is defined as a function named `new` in the class declaration. When you call the `new` constructor to create a class instance, the `new` function in the class declaration is automatically called. The code body of the `new` function can execute whatever code is required, for example, initialize class properties to required values. The function does not have a return type (this is derived from the class declaration), but otherwise follows all the normal rules of a SystemVerilog function. For example, the constructor can have multiple input arguments with default values, if required.

A SystemVerilog class declaration can only contain a single explicit constructor. SystemVerilog does not support subroutine overloading, which would allow multiple declarations of a task or function with the same name (e.g., `new`) but different numbers or types of arguments.

Example: Class Definition and Instantiation

```
class frame;
  logic [4:0] addr;
  logic [7:0] payload;
  logic parity = 0;

  function new (input int add, dat);
    addr = add;
    payload = dat;
    genpar();
  endfunction

  function void genpar();
    parity = ^{addr, payload};
  endfunction

  function logic [13:0] getframe();
    return({addr, payload, parity});
  endfunction
endclass
```

Note the following:

- Initial property value
- Constructor arguments are assigned to properties
- Constructor calls method to get updated parity value

```
logic [13:0] framedata;
frame one = new(3, 16);

initial begin
  ...
  @(negedge clk);
  framedata = one.getframe();
  ...
end
```



Here is a class example of a `frame` object with three properties and three methods (including an explicit constructor).

Class properties can be given initial values, just like module variables. In this example, the initial value on `parity` is redundant because the parity is updated in the constructor.

The `addr` and `payload` properties are initialized in an explicit constructor using the `add` and `dat` arguments. In the constructor, we also call the `genpar()` method to update the parity property to be consistent with the `addr` and `payload` values. The parity generation is defined as a separate method as this will be a common operation to be executed on the frame instances. In the declaration of `genpar()`, the return type of the function is `void` and no arguments are passed to the method. As the function is declared inside the class, local to the properties it needs to access, then, by convention, the properties are accessed directly and are not passed as method arguments.

The other class method, `getframe()`, is a “packing” method in that it packs all the properties of the class into a single vector for passing into the design.

Current Object Handle: `this`



IEEE 1800-2012 8.11

- Keyword `this` is a handle to the current class instance.
 - You can use it to reference class identifiers re-declared within a local scope.
 - It is meaningful only for nonstatic members.
- Here, `this.addr` is used to access the class property `addr`.
 - Otherwise, it is hidden by the function argument `addr`.
- Alternatively, do not reuse class property names as class method arguments.

```

class frame;
  logic [4:0] addr;
  logic [7:0] payload;
  logic parity;
  ...
  function new(input int addr, dat);
    this.addr = addr;
    payload = dat;
    genpar();
    ...
  endfunction
endclass

```

The `this` keyword is a handle to the current class instance.

In the example, in the constructor of `frame`, we have renamed the input argument `addr` to have the same name as the class property `addr`. Therefore in the constructor, any reference to `addr` accesses the function argument only, and the class property `addr` is hidden from view. By prefixing the identifier `addr` with the keyword `this`, we point the reference outside the constructor to the current class instance, and hence we can access the class property `addr`.

Alternatively, we could avoid using the same identifier for both the function argument and class property, and the need for `this` goes away.

In simple SystemVerilog class applications, `this` is rarely required, but in more complex class environments, such as in UVM (Universal Verification Methodology), the `this` reference is extensively used, specifically for class hierarchies using aggregate classes.

What Are Static Class Properties?



The Static Class Properties are class properties which are created using the keyword `static`.

A static property is shared by all instances of a class.

In the example, it is used to:

- Count the number of frames created.
- Create a unique identity for each.

```
class frame;
  static int frmcount;
  int tag;
  logic [4:0] addr;
  logic [7:0] payload;
  logic parity;

  function new(input int add, dat);
    addr = add;
    payload = dat;
    genpar();
    frmcount++;
    tag = frmcount;
  endfunction

  ...
endclass
```

```
frame f1 = new(1, 0);
frame f2 = new(3, 2);
```

f1
frmcount 1
tag 1
addr 1
payload 0
parity 1

Static property
shared by
both instances

f1
frmcount 2
tag 1
addr 1
payload 0
parity 1

f2
frmcount 2
tag 2
addr 3
payload 2
parity 1

Normally, each class instance has a separate, unique copy of every class property.

However, a property can be defined as `static`, which means that only one copy of the property exists and this copy is shared among all instances of the class. An update to a static property from one instance updates the property for all instances.

Here a static property is used to keep track of the number of frames created, and also to assign a unique identity to each frame. The static property `frmcount` is incremented in the class constructor and assigned to a nonstatic property `tag`.

When frame instance `f1` is created, `frmcount` is incremented to 1 and assigned to `f1.tag`. When instance `f2` is created, `frmcount` is incremented to 2, which sets both `f1.frmcount` and `f2.frmcount` to 2, and `f2.tag` is assigned to 2.

Static property `frmcount` initializes to 0 (being of type `int`), but could also be explicitly initialized in the declaration:

```
static int frmcount = 0;
```

Note that we do not want to reset `frmcount` in the class constructor. The aim is to keep an independent count of the number of frames created, so every time we create a frame, we want to increment the current frame count, not set it to zero.

What Are Static Class Methods?



When Methods are declared as `static`, they are subject to all the class scoping and access rules but behave like a regular subroutine that can be called outside the class, even with no class instantiation.

```
class frame;
  static int frmcount;
  int tag;
  logic [4:0] addr;
  logic [7:0] payload;
  logic parity;

  function new(input int add, dat);
    addr = add;
    payload = dat;
    genpar();
    frmcount++;
    tag = frmcount;
  endfunction

  static function int getcount();
    return (frmcount);
  endfunction
...
endclass
```

Only static properties or other static methods can be accessed.

Can be called even if no class instantiations exist.

- From class name using resolution operator `::`
- From any class handle.

```
frame f1, f2;
int frames;
initial begin
  frames = frame::getcount(); // 0
  frames = f2.getcount();      // 0

  f1 = new(3,4);
  f2 = new(5,6);
  frames = f2.getcount();      // 2
end
```

Resolution operator access

Handle access

25 © Cadence Design Systems, Inc. All rights reserved.

cadence

A static method knows nothing about the current object and so can access only static properties or other static methods.

The advantage of a static method is that it can be called even if there are no current instances of the class. A static method can be accessed from any handle of the class, regardless of whether the handle contains an instance, using normal method access or from the class name using the resolution operator (`::`):

<class_name>::

This example uses a static function to return the current frame count, which is a static property.

Some coding guidelines mandate the calling of static methods using resolution operator access from the class type name. This serves to clearly indicate the method is static, and aids readability. For example, at first glance the `getcount()` call from `f2` at the beginning of the initial block could be interpreted as a nonstatic method call from an object instance. The resolution operator syntax is unambiguous as a static method call.

What Is Class Aggregation?



Aggregation refers to a class which collects (aggregates) a number of other classes into one object.

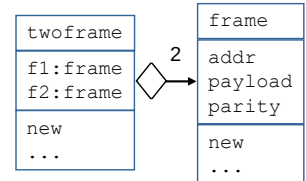
```
class frame;
  logic [4:0] addr;
  logic [7:0] payload;
  bit parity;

  function new(input int add, dat);
    addr = add;
    payload = dat;
    genpar();
    ...
  endfunction
  ...
endclass
```

```
class twoframe;
  frame f1;
  frame f2;

  function new(input int basea, d1, d2);
    f1 = new(basea, d1);
    f2 = new(basea+1, d2);
  endfunction
  ...
endclass
```

```
twoframe tf1 = new(2,3,4);
initial begin
  tf1.f2.addr = 4;
  $display("base %h", tf1.f1.addr);
  ...
end
```



26 © Cadence Design Systems, Inc. All rights reserved.

cadence

It is common for class objects to be building blocks for other classes. One way to achieve this is to declare class properties which are themselves handles of other classes. This is known as an aggregate or composite class (an aggregation, or collection, of other class instances).

We can use this to create a class which hold multiple frames with consecutive addresses.

Here class `twoframe` contains two object handles (`f1`, `f2`) of the class `frame` as properties. The constructor for `twoframe` should create the instances of `frame` by a call to the `frame` constructor. Without this call, the `frame` object handles in `twoframe` will be null. When we create an instance `tf1` of `twoframe`, the constructor creates the instances `f1` and `f2` of `frame` within the `tf1` instance.

To access the properties of `f1` within `tf1`, we must chain handles:

```
tf1.f1.addr
```

This accesses the property `addr` of the `frame` instance `f1`, which is a property of the `twoframe` instance `tf1`.

This is similar to a module hierarchy in Verilog. We can use a hierarchical pathname to access variables in a module-based hierarchy; likewise, we can use an handle pathname to access members of a class-based hierarchy.

What Is Class Inheritance?



Class Inheritance is extending a class by defining a derived class that inherits members[†] of the base class and may override inherited members and/or add new members.

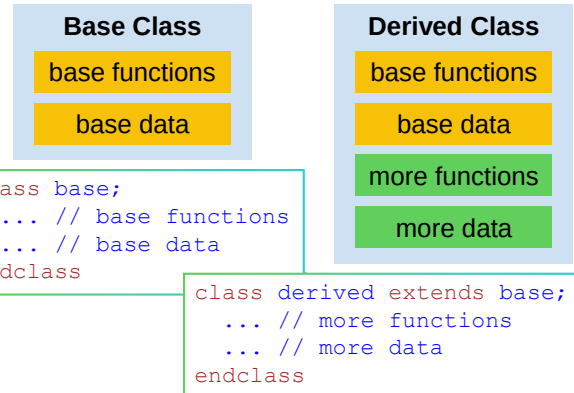
[†] Does **not** inherit constructors.

Why inheritance?

- Reusability makes the user more productive.
- Encourages use of proven software (why re-invent the basic wheel?).

Terminology

- Original class is called the superclass (OO) or base class (SV).
- New class is called the subclass (OO/SV) or derived class.
- SV supports only single inheritance: that is each subclass is derived from a single base class.
- Relationship “is a”.



Class Inheritance is extending a class by defining a derived class that inherits members of the base class and may override inherited members and may add new members.

Inheritance is essential to the object-oriented paradigm. Each initial description or refinement of the class is defined in one place and easily reused by further refinements to the class.

In an inheritance hierarchy, an inherited class is a “superclass” in object-oriented terms and a “base” class in SV terms, and the inheriting class is a “subclass” in object-oriented terms and a “derived” class in other language terms. Base classes and derived classes are relative terms, as a derived class can be a base for a further derivation.

SystemVerilog supports only single inheritance. That is each class can be derived from a single base class.

Example: Simple Inheritance

A sub-class inherits all the members of its parent class:

- Can add more members.
- Can re-declare (override) parent's methods.
- Constructors are not inherited.

Parent members are accessed as if they were members of the sub-class:

The sub-class constructor automatically calls the parent constructor:

- `frame.new()` called as the first line of `badtagframe` constructor.
- Why is this a problem?

```
badtagframe one = new(); //???
one.addr = 0;
```

Base-class

```
class frame;
  logic [4:0] addr;
  logic [7:0] payload;
  bit parity;

  function new(input int add, dat);
    addr = add;
    payload = dat;
    genpar();
  endfunction
  ...
endclass
```

Sub-class

```
class badtagframe extends frame;
  static int frmcount;
  int tag;

  function new();
    frmcount++;
    tag = frmcount;
  endfunction
  ...
endclass
```

 Error

An instance of the sub-class inherits all the members of its parent class and, by default, they can be referenced as if they were members of the sub-class.

A sub-class can also declare additional members and even override members from the parent class. An instance of the sub-class will, by default, always access its local overridden member declarations (if present) in preference to those from the parent class.

Hence using classes as building blocks for more specialized or generic classes is very easy.

The sub-class can include an explicit or default constructor, but the sub-class constructor always calls the parent class constructor as the first line. This is to make sure the properties inherited from the parent class are correctly initialized before the sub-class properties are processed. For example, in the code above, the compiler automatically inserts a call to the `frame` constructor *as the first line* of the `badtagframe` constructor. If you are using default constructors, this is no problem, but, in our example, the `frame` class has an explicit constructor with arguments which do not have default values. The call to the parent constructor inserted by the inheritance mechanism lacks the arguments required and so a compilation error is reported.

Example: Inheritance and Constructors

```

class frame;
  logic [4:0] addr;
  logic [7:0] payload;
  bit parity;

  function new(input int add, dat);
    addr = add;
    payload = dat;
    genpar();
  endfunction
  ...
endclass

```

Parent

The frame constructor requires arguments:

- Must be explicitly called to pass these arguments.
- Must be the first line of sub-class constructor to overwrite the implicit call.

super allows a sub-class to access parent members:

- Otherwise hidden by sub-class declarations.

```

class badtagframe extends frame;
  ...
  function new();
    super.new();
    frmcount++;
    tag = frmcount;
  endfunction
  ...

```

Sub-class

Automatically inserted



Error

```

class goodtagframe extends frame;
  ...
  function new(input int add, dat);
    super.new(add, dat);
    frmcount++;
    tag = frmcount;
  endfunction
  ...

```

Sub-class

First line explicit call overwrites implicit call

In this example, the frame constructor requires arguments to set the initial values of `addr` and `payload`. The implicit constructor call automatically inserted by the inheritance mechanism (derived from the `extends`) does not have these arguments, and so a compilation error results.

By explicitly adding the constructor call, we can overwrite the implicit call and pass the required arguments up to the `frame` constructor. However, in the `goodtagframe` class constructor, any reference to `new` would access the local constructor, creating a recursive function call. Nor can we use `this` as it would point to the `goodtagframe` instance. The keyword `super` allows us to access any member of the parent class which is otherwise hidden by being overridden in the sub-class. Therefore `super.new(add, dat)` allows us to call the `frame` constructor and pass the required arguments. Note that the `add` and `dat` arguments must be added to the `goodtagframe` constructor and passed up to the `frame` constructor in the `super` call.

An explicit `super.new` call must be the first line of the sub-class constructor. If we do not make it the first line, an implicit constructor call is added automatically. This means that the sub-class constructor cannot process any arguments before they are passed up to the parent class constructor.

Quick Reference Guide: Aggregation and Inheritance



Aggregation	- A class has properties which are instances of other classes: - Relationship “has a” - Similar to module instantiation - Each handle must be explicitly constructed.	
Inheritance	- Define a new sub-class by extending a parent class: - Relationship “is a” - Similar to <code>`include</code> - A sub-class inherits all members of the parent class except the constructor.	

30

© Cadence Design Systems, Inc. All rights reserved.

cadence

Aggregation refers to a class which collects (aggregates) a number of other classes into one object. There is no limit on the number of classes which can be aggregated, but each object handle must be explicitly constructed. This is referred to as a “has a” relationship, in that the class `twoframe` *has a* property `f1`, which is an instance of class `frame` and also *has a* property `f2`, which is also an instance of `frame`. In traditional Verilog, this is similar to module instantiation. Any instance of class `twoframe` instantiates two instances of `frame` `f1` and `f2` and to access the `frame` instances of `twoframe`, we use a hierarchical pathname.

Inheritance is a different way of using classes as building blocks, and is the basis of object-oriented design. Inheritance allows a class (sub-class) to extend another class (parent). We can use this to separate the tag functionality of the `frame` class from the packet properties. The `tagframe` class extends `frame`. The sub-class inherits all the base class members (`addr`, `parity` etc.) and can add new members (`frmcount`, `tag`) or override existing members. This is referred to as a “is a” relationship in that the sub-class `tagframe` *is a* `frame` class, albeit with additional members. In traditional Verilog, this is similar to an ``include` directive in that the declarations from the parent class appear as if they were declared in the sub-class.

Crucially, the sub-class is still compatible with the base class and handles of both classes can be mixed. SystemVerilog only allows single inheritance in that a sub-class can only extend from one base class.

The base class is also called the super class. The sub-class is also called the derived class or child class.

Multi-Layer Inheritance



You can layer inheritance to multiple generations. Each new level inherits the members of the previous levels and can override any of these members and can add new members.

`errframe` is a sub-class of `tagframe`.

- Induces parity errors.
- Has access to `tagframe` and `frame` members.
- Constructors are explicit and cascaded.

You can only pass arguments one level at a time:

- `super.super.new()` is not allowed.

```

class frame;                                Base class
  logic [4:0] addr;
  logic [7:0] payload;
  bit parity;

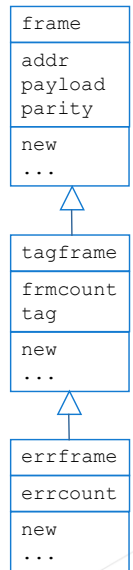
  function new(int add, int dat);
    ...
endclass

class tagframe extends frame;               Sub-class
  static int frmcount;
  int tag;

  function new(input int add, dat);
    super.new(add, dat);
    ...
endclass

class errframe extends tagframe;           Sub-class
  static int errcount;

  function new(input int add, dat);
    super.new(add, dat);
  endfunction
  ...
endclass
  
```



Inheritance can, and often is, layered to many levels. Each new level inherits the members of the previous levels and can override any of these members and can add new members.

In this example, `errframe` is a specialized version of `tagframe` which inserts errors in the frame data. It has its own methods and a static property `errcount`.

`tagframe` is a parent class of `errframe` and a sub-class of `frame`. `frame` is the parent class of `tagframe`. The highest parent class at the base of the inheritance hierarchy is referred to as the base class.

Because the base class constructor is parameterized, we must explicitly define constructors for each sub-class and pass the parameters up the inheritance hierarchy. Note that `super` refers to the immediate parent class only. There is no way to reach higher (`super.super.new` is not allowed).

What Is Data Hiding and Encapsulation?



To restrict the access to class properties and methods from outside the class by hiding their names, System Verilog (SV) provides local and protected identifiers.

By default, class members are visible externally and in all sub-classes.

Two keywords hide members:

- local – Only visible inside the class.
- protected – Visible inside class and any sub-classes.
 - That is, can be inherited.

```

class frame;
  local logic [4:0] addr;
  local logic [7:0] payload;
  protected bit parity;

  function void tag;
    int tag;
  endfunction
endclass

class tagframe extends frame;
  local static int frmcount;
  ...
endclass

class errframe extends tagframe;
  local static int errcount;
  ...
  function void add_error();
    parity = ~parity;
    errcount++;
  endfunction

  static function int geterr();
    return (errcount);
  endfunction
endclass
  
```

Diagram labels: Parent class (frame), Sub-class (tagframe), Sub-class (errframe).

```

errframe one = new(addr1, data1);
initial begin
  one.errcount = 0;
  one.parity = 1;
  one.add_error();
  no_errs = one.errcount;
  no_errs = errframe::geterr();
end
  
```



Which of the lines above generate errors?

Question

Remember, protected members are inherited, whereas local members are not.

What Are Parameterized Classes?



The SystemVerilog parameter mechanism is used to parameterize a class. It allows user to define a generic class whose objects can be instantiated to have different array sizes or data types.

```
class stack #(parameter type sign = int);
  local sign data[100];
  static int depth;

  task push(input sign indat... );
    ...
  endtask
  task pop( output sign outdat ... );
    ...
  endtask
endclass

// int stack (default)
stack intstack = new();

// 8-bit vector stack
stack #(logic[7:0]) bytestack = new();
```

- Classes can have parameters just like modules and interfaces.
- Parameters can be types as well as values.
 - A type parameter creates a "class template."
- Parameter values can be overridden for individual instances.
 - Methods using type parameters must work for all expected type overrides.
- Each new parameter value effectively creates a new class declaration or class specialization.
 - Each class specialization has a separate set of static properties.

33

© Cadence Design Systems, Inc. All rights reserved.



Classes can have parameters, just like modules and interfaces, and these parameters can be mapped to different values for each instance of a class. For example, this would allow the user to change the size of an array property for each instance of a class.

Verilog-2001 allows type parameters, and type parameters can also be used with classes. A class with a type parameter is known as a template class, and typically defines some functionality (here a stack) which can be carried out on many different types. An issue with template classes is defining methods so they work with all combinations of expected types, including SystemVerilog types such as `struct` and `enum`.

Each new value or type for a parameter effectively creates a new class declaration (or class specialization). One consequence is that each class specialization has a separate set of static properties.

Note that there are alternative, and better, constructs in SystemVerilog for implementing stack-like functionality, such as a `queue` dynamic array, where stack elements are of the same type, or a `mailbox` where the elements can be of different types.

Module Summary

With the SystemVerilog Object-Oriented testbench features, you can:

- Declare dynamic data types:
 - Create and destroy objects during simulation
- Declare hierarchical data types:
 - Building up “layers” of types promotes reuse
- Encapsulate data with protected and local class members:
 - Hiding your data from external methods simplifies maintenance of the type
- Raise the level of abstraction for building testbenches:
 - You can develop and debug the test environment more quickly



This page does not contain notes.

Quiz



What is the role of a class constructor?

- The class constructor creates an instance and initializes the class properties.



What is the difference between a static and non-static property?

- One instance of a nonstatic property exists for each instance of the same class type.
- One instance of a static property exists for all instances of the same class type.



What is the difference between a static and non-static method?

- A static method of a class can access only the static members of the class.



What is inheritance?

- A sub-class inherits the members of its super class, and can add new members and re-declare those of the parent class.



What is the difference between a protected and a local class member?

- A sub-class method can access a protected member of its super class.
- Only class member methods can access local members of a class.

This page does not contain notes.